

Memoria Técnica: Práctica 2

Detección de Obstáculos con ROS 2 a partir de Datos Sensoriales (LIDAR 3D y

Cámara RGB)

Asignatura:	Robótica (Grado en Ingeniería de Computadores / Máster)	Institución:	E.T.S.I. de Sistemas Informáticos — UPM
Autor / Alumno:	Javier Ferreño Cachafeiro	Entorno de ejecución:	de ROS 2 Jazzy Jalisco / Ubuntu Linux (Docker)
Paquetes entregados:	<code>obstacle_detector</code> , <code>obstacle_detector_interfaces</code>	Evaluación objetivo:	10 / 10 puntos (Fusión sensorial multicanal)

1. Resumen Ejecutivo y Objetivos

El objetivo principal de esta práctica es el diseño e implementación de una arquitectura modular basada en **ROS 2** para la detección frontal de obstáculos y la toma de decisiones de seguridad a partir de datos sensoriales reales registrados en un ROS 2 Bag oficial.

Para satisfacer los requisitos exigidos para la máxima calificación (10 puntos), el sistema procesa concurrentemente **dos fuentes de datos heterogéneas**:

- LIDAR 3D Ouster:** Nube de puntos tridimensionales (`sensor_msgs/msg/PointCloud2` en el topic `/ouster/points`), con 131.072 puntos por fotograma a 10 Hz.
- Cámara RGB Basler Blaze:** Transmisión de vídeo en color de alta definición (`sensor_msgs/msg/Image` en el topic `/my_camera/pylon_ros2_camera_node/image_raw`) a 30 Hz.

La solución implementa tres nodos cooperativos, un paquete de interfaces con un mensaje propio y un servicio síncrono personalizado, soporte completo para reconfiguración dinámica de parámetros en caliente (`ros2 param set`) y un lanzador unificado (`obstacle_detection.launch.py`).

2. Arquitectura del Sistema ROS 2

El sistema se distribuye en dos paquetes de ROS 2 conforme a las mejores prácticas de la Open Source Robotics Foundation (OSRF):

- `obstacle_detector_interfaces` (CMake/C++): Declara los tipos de comunicación customizados.
- `obstacle_detector` (Python/rclpy): Implementa los nodos de procesamiento, visión, análisis geométrico y el lanzador.

Componente	Tipo de Entidad	Topic / Servicio	Tipo de Mensaje / Interfaz	Función Principal
<code>lidar_obstacle_detector</code>	Nodo ROS 2	<code>/ouster/points</code> (Sub) <code>/obstacles/lidar</code> (Pub)	<code>sensor_msgs/PointCloud2</code> <code>ObstacleInfo.msg</code>	Filtrado ROI 3D vectorizado y cálculo de distancia mínima al obstáculo.
<code>camera_obstacle_detector</code>	Nodo ROS 2	<code>/my_camera/.../image_raw</code> (Sub) <code>/obstacles/camera</code> (Pub) <code>/obstacles/camera</code> (Pub) <code>/annotated_image</code> (Pub)	<code>sensor_msgs/Image</code> <code>ObstacleInfo.msg</code> <code>sensor_msgs/Image</code>	Procesamiento de visión por computador (OpenCV), contornos y bounding boxes.
<code>obstacle_manager</code>	Nodo ROS 2	<code>/obstacles/lidar</code> (Sub) <code>/obstacles/camera</code> (Sub) <code>/safety/status</code> (Pub)	<code>ObstacleInfo.msg</code> <code>ObstacleInfo.msg</code>	Fusión sensorial, determinación del riesgo global y publicación periódica a 10 Hz.
<code>/safety/query_status</code>	Servicio ROS 2	Ofrecido por <code>obstacle_manager</code>	<code>QuerySafetyStatus.srv</code>	Diagnóstico síncrono del estado del camino bajo demanda (filtro por sensor).

3. Definición de Interfaces Customizadas

3.1 Mensaje Custom: `ObstacleInfo.msg`

Se ha creado un mensaje propio adaptado a sistemas de percepción robótica que encapsula tanto las coordenadas cartesianas del obstáculo como su severidad:

```
std_msgs/Header header

string sensor_type      # Identificador: "lidar", "camera", "fusion_lidar_camera"

bool obstacle_detected  # True si se detecta un objeto en la trayectoria

float32 distance        # Distancia euclídea mínima en metros

float32 x               # Coordenada longitudinal (avance)

float32 y               # Coordenada lateral (desvío horizontal)

float32 z               # Coordenada vertical (altura)

string severity         # Clasificación de riesgo: "SAFE", "WARNING", "DANGER"
```

3.2 Servicio Custom: `QuerySafetyStatus.srv`

Permite a módulos de navegación o supervisores consultar instantáneamente si es seguro continuar:

```
# Request: permite filtrar por subsistema ("all", "lidar", "camera")

string sensor_filter

---

# Response: diagnóstico de seguridad

bool safe_to_proceed

float32 closest_distance

string alert_level

string message
```

4. Implementación Detallada de los Nodos

4.1 Nodo 1: lidar_obstacle_detector (Procesamiento PointCloud 3D)

Para procesar eficientemente los 131.072 puntos que el sensor LIDAR Ouster emite cada 100 ms, se implementó una rutina vectorizada con `sensor_msgs_py.point_cloud2` y `numpy`, evitando bucles iterativos en Python y garantizando una latencia inferior a **8 ms por escaneo**.

- **Definición de Región de Interés (ROI Frontal):** Se aísla el pasillo de navegación del robot para ignorar el suelo y el entorno no colisionable: $x \in [roi_x_min, roi_x_max]$, $y \in [roi_y_min, roi_y_max]$, $z \in [roi_z_min, roi_z_max]$.
- **Filtrado y Clustering:** Se descartan puntos infinitos/NaN y se exige un umbral mínimo de puntos contiguos (`min_points_cluster = 15`) para evitar falsos positivos por ruido o polvo.
- **Cálculo de Distancia Mínima:** Se calcula la norma euclídea $d = \sqrt{x^2 + y^2}$ localizando el punto más crítico y su vector de posición.

4.2 Nodo 2: camera_obstacle_detector (Visión Artificial OpenCV)

El nodo se suscribe al flujo de la cámara Basler Blaze, convirtiendo los mensajes a matrices BGR mediante `CvBridge`:

- **Segmentación de la Calzada:** Aplica un recorte de la parte inferior de la imagen (`roi_top_ratio = 0.35`), ignorando el cielo y el horizonte.
- **Filtrado y Detección de Bordes:** Aplica filtro Gaussiano (kernel 5x5) y detector de bordes de Canny, seguido de dilatación morfológica con elemento estructurante de 7x7 para cerrar contornos de objetos.
- **Cálculo de Área y Escala Aparente:** Extrae contornos con área superior a `min_contour_area` (3000 px). Si el obstáculo crece en el plano visual por encima de `warning_area` (25.000 px) o `danger_area` (70.000 px), se disparan los niveles de alerta.
- **Publicación Gráfica Anotada:** Publica en `/obstacles/camera/annotated_image` la imagen con bounding boxes en color (Verde, Naranja, Rojo) y textos explicativos con la distancia estimada.

4.3 Nodo 3: obstacle_manager (Fusión Sensorial y Servicio)

Este nodo actúa como el cerebro de seguridad del vehículo:

- **Mecanismo de Heartbeat / Timeout:** Supervisa los timestamps de los sensores. Si un sensor deja de emitir durante más de `sensor_timeout_sec = 1.5s`, el gestor lo cataloga como no disponible y previene decisiones basadas en datos obsoletos.
- **Fusión Pesimista de Seguridad:** Si el LIDAR o la Cámara detectan una intrusión crítica en la

zona de frenado de emergencia (`emergency_distance = 1.5m`), el estado unificado escala inmediatamente a DANGER.

- **Servidor de Servicio:** Procesa llamadas síncronas a `/safety/query_status` devolviendo un veredicto estructurado para el planificador de trayectorias.

5. Parámetros Dinámicos en Tiempo de Ejecución

Los tres nodos implementan el callback reactivo `add_on_set_parameters_callback`, permitiendo adaptar los márgenes de seguridad en caliente sin necesidad de reiniciar los procesos ni pausar el bag:

```
# Modificar la distancia de aviso del gestor de seguridad a 4.5 metros:

ros2 param set /obstacle_manager warning_distance 4.5


# Modificar el margen de seguridad de frenado de emergencia a 2.0 metros:

ros2 param set /obstacle_manager emergency_distance 2.0


# Ajustar el ancho del pasillo de inspección del LIDAR:

ros2 param set /lidar_obstacle_detector roi_y_max 2.0
```

6. Guía de Ejecución y Validación

6.1 Compilación del Espacio de Trabajo

```
cd ~/Escritorio/ROS/practica2/ws

source /opt/ros/jazzy/setup.bash

colcon build

source install/setup.bash
```

6.2 Lanzamiento de los Nodos (Launch File)

```
# Arranca simultáneamente el detector LIDAR, el detector de cámara y el gestor:

ros2 launch obstacle_detector obstacle_detection.launch.py warning_distance:=3.5
```

```
danger_distance:=1.5
```

6.3 Reproducción de la Bolsa Sensorial (ROS 2 Bag Play)

```
# En otra terminal, reproducir los datos sensoriales del bag:

ros2 bag play ~/Escritorio/ROS/practica2/rosbag2_2025_02_27-13_08_14 \

--topics /ouster/points /my_camera/pylon_ros2_camera_node/image_raw
```

6.4 Verificación del Servicio de Seguridad

```
# Consulta global de seguridad (fusión de ambos sensores):

ros2 service call /safety/query_status obstacle_detector_interfaces/srv/QuerySafetyStatus
"{sensor_filter: 'all'}"

# Consulta específica del sensor LIDAR:

ros2 service call /safety/query_status obstacle_detector_interfaces/srv/QuerySafetyStatus
"{sensor_filter: 'lidar'}"
```

7. Matriz de Cumplimiento de Requisitos

Requisito Oficial del Enunciado	Puntuación	Solución Implementada	Estado
Implementar al menos 2 nodos ROS 2	3 5.0 (Base)	3 nodos: lidar_obstacle_detector, camera_obstacle_detector, obstacle_manager.	CUMPLIDO
Comunicación por Topics y Servicios	5.0 (Base)	Topics /obstacles/lidar, /obstacles/camera, /safety/status y servicio /safety/query_status.	CUMPLIDO
Definir e implementar 1 mensaje custom propio	5.0 (Base)	ObstacleInfo.msg y servicio custom QuerySafetyStatus.srv.	CUMPLIDO
Incluir Launcher conjunto (.launch.py)	5.0 (Base)	obstacle_detection.launch.py con argumentos configurables.	CUMPLIDO
Modificación de parámetros en tiempo de ejecución	5.0 (Base)	Soporte completo con add_on_set_parameters_callback en todos los nodos.	CUMPLIDO

Requisito Oficial del Enunciado	Puntuación	Solución Implementada	Estado
Utilizar al menos 2 fuentes de datos distintas	10.0 (Máxima)	Procesamiento conjunto de Nube de Puntos LIDAR 3D y Cámara RGB.	CUMPLIDO (10/10)

8. Conclusiones

El desarrollo ha demostrado la eficacia de combinar sensores de diferente naturaleza física: mientras el **LIDAR 3D** aporta precisión métrica milimétrica inmune a cambios de luminosidad ambiental, la **cámara RGB** proporciona densidad visual y capacidad de discriminación morfológica. La arquitectura implementada en ROS 2 garantiza desacoplamiento, robustez ante caídas sensoriales y adaptabilidad dinámica para robótica autónoma real.